Data Governance Copilot

Interview Question Bank

Topic 10 — Retry Logic & Error Handling

12 questions · 4 sections · Exponential Backoff · @with_retry · retry_agent_call · AgentResult · Error Hierarchy · Circuit Breaker

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

---

Retry Fundamentals & Backoff Strategy

Q01 Foundational

Explain exponential backoff. What is it, why is it used instead of fixed-interval retries, and what are the specific backoff values in your project?

Hint: Calculate the actual wait times and explain why each interval is appropriate.

Key points to cover:

- Exponential backoff: wait time doubles after each failed attempt — backoff_factor * 2^attempt
- Your config: max_retries=3, backoff_factor=1.0 → waits: 1s (attempt 0), 2s (attempt 1), 4s (attempt 2)
- Total worst-case wait before final failure: 1 + 2 + 4 = 7 seconds before giving up
- Why not fixed interval (e.g. retry every 1s): if 100 clients all retry every 1s simultaneously, the failing service receives the same thundering herd that caused the failure
- Exponential backoff spreads retries over time — reduces load on recovering services; different clients reach different backoff intervals naturally
- Jitter improvement: add random.uniform(0, backoff) to spread retries further — prevents correlated retry storms across ECS tasks
- Why 3 retries: covers transient network blips (1-2s), Databricks SQL warehouse cold start (~3-5s), Groq API rate limit window — without retrying indefinitely
- Retry budget: 7s total retry wait + original request time — acceptable for a governance query; user expects 5-15s response time

```
# retry.py backoff calculation import time, random def exponential_backoff(attempt: int, backoff_factor: float = 1.0, jitter: bool = True) -> float: wait = backoff_factor * (2 ** attempt) # 1s, 2s, 4s if jitter: wait += random.uniform(0, wait * 0.1) # ±10% jitter return wait # Attempt 0: wait 1.0-1.1s # Attempt 1: wait 2.0-2.2s # Attempt 2: wait 4.0-4.4s # Attempt 3: FAIL — raise final exception
```

Q02 Foundational

Explain the difference between @with_retry decorator and retry_agent_call() function. When would you use each?

Hint: Your project has two retry mechanisms — they serve different purposes.

Key points to cover:

- @with_retry(max_retries=3, backoff_factor=1.0): a decorator applied at definition time — wraps any function with retry logic
- Usage: applied directly to agent node functions or service methods — declarative, clean, no call-site changes needed
- retry_agent_call(agent.execute, request, max_retries=3): a function called at invocation time — more flexible, allows per-call retry configuration
- Usage: called inside information_node, knowledge_node, metadata_node — allows passing the specific AgentRequest and catching AgentResult failures
- Key difference: @with_retry retries on exceptions; retry_agent_call can also retry on AgentResult(success=False) — a soft failure that doesn't raise
- Why both: @with_retry is for infrastructure-level retries (network errors, timeouts that raise exceptions); retry_agent_call handles business-level failures (agent returned success=False)
- Applied to: information_node, knowledge_node, metadata_node — the three agents with external dependencies (Databricks, pgvector, Collibra)
- NOT applied to: capacity_node (Jira writes must not be retried blindly — risk of duplicate tickets), auto_ticket_node, synthesizer_node

Q03 Intermediate

Walk through the @with_retry decorator implementation. What exceptions does it catch, what does it do on each attempt, and what happens after max_retries is exhausted?

Hint: Implement the decorator mentally — explain every branch.

Key points to cover:

- Decorator factory: @with_retry(max_retries=3, backoff_factor=1.0) returns a decorator that wraps the target function
- On each call: try the wrapped function; if it succeeds, return immediately
- On exception: log the attempt number and exception message; calculate backoff wait; sleep(wait); try again
- Exception types caught: Exception (broad catch) — but should be narrowed to transient errors: ConnectionError, TimeoutError, HTTPError (429, 503)
- Should NOT catch: ValueError, TypeError, KeyError — these are programming errors; retrying won't fix them
- After max_retries exhausted: re-raise the last exception — caller must handle it
- In your nodes context: information_node catches the final exception and returns AgentResult(success=False, error=str(e))
- Logging on each retry: structured log with attempt number, exception type, wait time — critical for diagnosing flaky services in LangSmith/CloudWatch

```
def with_retry(max_retries=3, backoff_factor=1.0): def decorator(fn): @functools.wraps(fn) # preserve fn.__name__ for tracing def wrapper(*args, **kwargs): last_exc = None for attempt in range(max_retries + 1): try: return fn(*args, **kwargs) except (ConnectionError, TimeoutError, HTTPError) as e: last_exc = e if attempt == max_retries: break # exhausted — fall through wait = backoff_factor * (2 ** attempt) logger.warning(f'Retry {attempt+1}/{max_retries} ' f'after {wait}s: {e}') time.sleep(wait) raise last_exc # re-raise final exception return wrapper return decorator
```

AgentResult Failure Path & Error Propagation

Q04 Intermediate

Explain the AgentResult failure path. How does a failed agent affect the rest of the graph and the final response to the user?

Hint: Trace what happens from agent failure all the way to final_summary.

Key points to cover:

- Agent execution fails after all retries: node catches the exception, returns AgentResult(success=False, data={}, error='Connection timeout after 3 retries')
- Node appends AgentResult to state['agent_results'] — even failed results are accumulated (Annotated list)
- Node also appends error string to state['errors'] — a separate accumulator for the error list
- _after_agent conditional: continues to the next agent regardless of the previous agent's success — partial results are better than no results
- synthesizer_node receives all agent_results including failed ones — it inspects success field on each
- Synthesizer prompt: 'InformationAgent failed: Connection timeout. KnowledgeAgent returned: [result]. MetadataAgent returned: [result]. Synthesize what is available.'
- final_summary acknowledges the partial failure: 'Based on available data (Databricks unavailable): ...' — honest, useful response
- confidence in state is lowered when agents fail — surfaced in API response so client UI can show 'partial results' warning

Q05 Advanced

CapacityAgent (Jira) is NOT wrapped with retry. Why is blindly retrying a Jira ticket creation dangerous? What pattern do you use instead?

Hint: Write operations have different retry semantics than read operations.

Key points to cover:

- Jira REST API: POST /rest/api/3/issue creates a ticket — not idempotent by default; two identical POST calls create two tickets
- Retry scenario: first POST succeeds but response times out (network glitch) — retry fires, creates a second duplicate ticket
- Impact in governance: two identical incidents for the same data quality issue — duplicate Jira tickets confuse the ops team
- Pattern instead: idempotency key — include a unique idempotency_key in the Jira request (custom field or issue summary hash)
- Before creating: search Jira for existing tickets with the same idempotency_key — if found, skip creation and return existing ticket ID
- Alternative: Jira supports idempotency via X-Atlassian-Token header in some API versions — use if available
- HITL gate reinforces this: auto_ticket_node only fires on approved=True — human approval is itself an idempotency control (user won't approve twice for the same event)
- Lesson: retry is safe for idempotent operations (GET, PUT with same payload); dangerous for non-idempotent (POST creating resources)

Q06 Advanced

Three agents run sequentially: InformationAgent succeeds, KnowledgeAgent fails permanently (all retries exhausted), MetadataAgent succeeds. What does state look like at synthesizer_node and what does the user get?

Hint: Trace the exact state contents after partial failure.

Key points to cover:

- state['agent_results'] after all agents: [AgentResult(success=True, data={metrics}), AgentResult(success=False, error='pgvector timeout'), AgentResult(success=True, data={metadata})]
- state['errors']: ['KnowledgeAgent: pgvector timeout after 3 retries']
- state['sources']: ['databricks://table/bookings', 'collibra://asset/retention'] — KnowledgeAgent contributes nothing (no RAG docs retrieved)
- synthesizer_node prompt includes: all three results; error list; instruction to synthesise only from successful agents
- LLM synthesises: 'Based on Databricks metrics and Collibra metadata (governance documents unavailable due to vector store timeout): retention rate is 87% ...'
- final_summary transparently acknowledges the gap — user knows RAG knowledge is missing from this answer
- confidence is reduced: e.g. 0.65 instead of 0.92 — reflects that one of three knowledge sources was unavailable
- Monitoring signal: KnowledgeAgent failure logged with query_id — CloudWatch metric 'agent_failure_rate' alerts if pgvector failures spike

---

Error Hierarchy & Classification

Q07 Intermediate

Describe the error hierarchy in core/logging_utils.py. What are the custom exception classes and why have a hierarchy instead of raising generic Exception?

Hint: Think about catch specificity and operational alerting.

Key points to cover:

- Custom exception hierarchy: base DGCException → AgentException, ServiceException, GuardrailException, ConfigurationException
- AgentException: raised when an agent's execute() method encounters a business logic failure (not a transient network error)
- ServiceException: raised by service layer (DatabricksService, CollibraService) for infrastructure failures — wraps underlying network/auth errors
- GuardrailException: raised by guardrails.py when a check fails — distinct from agent failures for audit routing
- ConfigurationException: raised at startup if required env vars are missing — fast-fail with clear message
- Why hierarchy vs generic Exception: catch blocks can be specific — retry_agent_call catches ServiceException (transient, retryable) but not AgentException (business logic, not retryable)
- Alerting: CloudWatch alarm on ServiceException rate (infrastructure issue); different alarm on AgentException rate (model/logic issue) — different on-call runbooks
- Structured logging: each exception class includes a code field ('SVC_TIMEOUT', 'AGT_VALIDATION') — log aggregation and dashboards filter by code

```
# Error hierarchy class DGCException(Exception): def __init__(self, message, code=None): super().__init__(message) self.code = code class ServiceException(DGCException): pass # retryable class AgentException(DGCException): pass # not retryable class GuardrailException(DGCException): pass # audit path class ConfigurationException(DGCException): pass # startup # Selective retry try: result = service.query_metrics(sql, params) except ServiceException as e: # transient — retry raise except AgentException as e: # logic error — don't retry return AgentResult(success=False, error=str(e))
```

Q08 Intermediate

What errors should NOT be retried and how do you distinguish them from transient errors in your retry logic?

Hint: Retrying the wrong errors wastes time and can make things worse.

Key points to cover:

- Do NOT retry — programming errors: TypeError, ValueError, AttributeError — retrying identical input will always fail identically
- Do NOT retry — auth failures: HTTP 401, 403 — credentials are wrong; retrying won't fix them; immediate fail + alert
- Do NOT retry — client errors: HTTP 400 Bad Request — the request itself is malformed; retrying sends the same bad request
- Do NOT retry — budget exceeded: TokenBudgetExceeded — retrying consumes more budget against an already-exceeded limit
- Do NOT retry — validation errors: Pydantic ValidationError from structured output — LLM returned wrong schema; retry may help but is not guaranteed; limit to 1 retry
- DO retry — transient: HTTP 429 Rate Limited, 503 Service Unavailable, ConnectionTimeout, ReadTimeout — recoverable with backoff
- DO retry — intermittent: pgvector connection reset, Redis connection refused (brief), Databricks SQL warehouse warming up
- Implementation: @with_retry should catch a specific tuple of retryable exception types, not bare Exception
- Off-by-one in retry counts is Bug pattern in your project: max_retries=3 means 3 retries (4 total attempts including original) — document this clearly

Q09 Gotcha

Your retry logic has max_retries=3. A teammate says 'that means 3 attempts total'. You say it means 4. Who is right and why does this matter?

Hint: This is Bug pattern from your project — off-by-one in retry count interpretation.

Key points to cover:

- Depends entirely on the implementation — this is the off-by-one ambiguity in retry semantics
- Interpretation A: max_retries=3 means 3 RETRIES after the original attempt → 4 total attempts (1 original + 3 retries)
- Interpretation B: max_retries=3 means 3 TOTAL attempts → 2 retries after the original
- Your project (retry.py): for attempt in range(max_retries + 1) — this is interpretation A: 4 total attempts
- Why it matters for backoff: interpretation A gives backoff waits [1s, 2s, 4s]; interpretation B gives [1s, 2s] — 7s vs 3s total wait
- Why it matters for cost: 4 LLM calls vs 3 LLM calls per failed request — multiplied across many concurrent failures
- Bug scenario: test asserts exactly 3 LLM calls on failure but implementation makes 4 — test passes if mock is permissive, silently wrong
- Fix: use explicit naming — max_attempts=4 is unambiguous; or document in docstring: 'max_retries: number of retry attempts AFTER the initial call'
- This is listed in your project bug patterns — a real source of test confusion during development

---

Production Resilience & Design Depth

Q10 Advanced

Your retry adds up to 7 seconds of backoff delay in the worst case. For a FastAPI endpoint with a 30-second timeout, this is fine. But what happens in a Lambda or a Teams webhook with a 3-second response requirement?

Hint: Retry strategy must be adapted to the caller's timeout budget.

Key points to cover:

- Teams webhook: Microsoft expects a response within 5 seconds or marks the request as failed and retries from their side
- If your 3-retry backoff takes 7s, Teams sends a duplicate webhook → your bot processes the same message twice → duplicate Jira tickets
- Lambda: 15-minute timeout is generous but cold start + 7s backoff on every invocation can cascade into runaway costs
- Adaptation for time-constrained callers: reduce max_retries=1 with backoff_factor=0.5 → 0.5s wait → 2 total attempts, 0.5s max delay
- Alternative: async retry with timeout budget — retry_with_budget(fn, budget_ms=3000) — retries only if remaining budget allows
- Teams bot specific: respond with 200 immediately (fire-and-forget acknowledgment), process asynchronously, send result via proactive message
- Proactive message pattern: Teams bot sends acknowledgment in <1s; background task runs _run_graph() with full retry; posts result as a new message
- This is why your Teams bot uses Adaptive Cards — the async result can be posted as a new card in the conversation after full processing

---

Q11 Advanced

Describe a circuit breaker pattern. Does your project implement one and should it?

Hint: Circuit breakers are a step beyond retry — understand the distinction.

Key points to cover:

- Circuit breaker: tracks failure rate for a service; if failures exceed threshold, 'opens' the circuit and immediately rejects calls without attempting
- States: CLOSED (normal, calls pass through) → OPEN (failures exceeded threshold, calls rejected instantly) → HALF-OPEN (probe call to test recovery)
- Difference from retry: retry waits and tries again; circuit breaker stops trying entirely until the service shows signs of recovery
- Your project: no explicit circuit breaker — retry with backoff is the only resilience pattern for agent calls
- Should it? Yes, for high-volume production: if Databricks SQL warehouse is down for 10 minutes, every query retries 4 times with 7s backoff — massive queued load
- With circuit breaker: after 5 consecutive Databricks failures, open the circuit — all subsequent information_node calls immediately return AgentResult(success=False, error='Circuit open: Databricks unavailable')
- Implementation options: tenacity library (Python) has circuit breaker; or use Redis to track failure counts across ECS tasks
- Half-open probe: every 60 seconds, attempt one Databricks call — if it succeeds, close the circuit; if it fails, keep open
- For your project: a per-service circuit breaker in factory.py would be the right place — each IDataService, IMetadataService gets its own breaker

---

Q12 Design

An interviewer asks: 'You retry at the agent node level. Why not retry at the LangGraph graph level — just re-run the entire graph on failure?' Defend your choice.

Hint: This tests understanding of retry granularity and its cost implications.

Key points to cover:

- Graph-level retry: if any node fails, re-run entire graph from START — simplest implementation, hardest on cost
- Problem 1: redundant work — if information_node and knowledge_node succeed but metadata_node fails, graph-level retry re-runs all three
- Problem 2: LLM cost: supervisor_node re-runs intent classification (another LLM call); synthesizer re-runs even though agent_results are identical
- Problem 3: cache interaction: @cached_node returns immediately for information_node and knowledge_node on retry — somewhat mitigates redundant work, but metadata_node miss still triggers full Collibra call
- Problem 4: HITL state: if auto_ticket_node set pending_action before the failure, graph-level retry resets the state — HITL approval is lost
- Agent-level retry advantage: retry only the failing operation; successful agents' results are preserved in state across the sequential run
- Agent-level retry advantage: retry budget is tight (7s) and localised — graph-level retry multiplies that by the number of nodes
- Correct answer: agent-level retry for transient failures; graph-level retry (via new HTTP request with same thread_id) only for complete system failures where the user explicitly retries
- LangGraph checkpoint enables this: user retries → new graph.ainvoke() loads checkpoint → state includes previous successful agent_results — effectively resumes from failure point

---